



PROJECT REQUIREMENTS SPECIFICATION

A novel method for video manipulation and compilation using
Big Data Technologies

UE21CS390A – Project Phase – 1

Submitted by:

NAME	SRN
Abhishek V Sagarnal, Divya D, Pradyun P Rao, Sarathak S Kumar	PES2UG21CS023 PES2UG21CS165 PES2UG21CS378 PES2UG21CS482

Under the guidance of

Dr. Chandrashekhar P Chavan

Designation
PES University

January - May 2024

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
FACULTY OF ENGINEERING
PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

TABLE OF CONTENTS

1. Introduction	3
1.1 Project Scope and Motivation	3
2. Literature Survey or Existing System	3
3. Product Perspective	3
3.1 Product Features	3
3.2 User Classes and Characteristics	3
3.3 Operating Environment	3
3.4 General Constraints, Assumptions and Dependencies	3
3.5 Risks	4
4. Functional Requirements	4
5. External Interface Requirements	4
5.1 User Interfaces	4
5.2 Hardware Requirements	4
5.3 Software Requirements	4
5.4 Communication Interfaces	5
6. Non-Functional Requirements	5
6.1 Performance Requirements	5
6.2 Safety Requirements	5
6.3 Security Requirements	5
7. Other Requirements	5
Appendix A: Definitions, Acronyms and Abbreviations	5
Appendix B: References	5

1. Introduction

[Introduce the document; describe the requirements of the product specified in this document.]

This document is an overview into the novel method we are proposing for video manipulation and compilation using Big Data Technologies

1.1. Project Scope

The purpose of this project is to provide a fast and efficient means to process high-quality, next-generation video formats in a parallelized computing form. By leveraging Big Data Technologies, the system will address the challenges of handling high bandwidth, large sizes of data, and high speeds of processing.

- Benefits:
 1. Improved video processing speed and efficiency.
 2. Facilitated rapid innovation in video technology.
 3. Reduced learning curve for implementing advanced video processing techniques.

- Objectives:
 1. Develop a system architecture for video manipulation and compilation using Big Data Technologies.
 2. Implement algorithms for processing of videos.
 3. Try to ensure hardware standardization for consistent video processing performance.

- Goals:
 1. Achieve time-measurable processing of high-quality videos.
 2. Reduce the time and expertise required for video processing, editing, and streaming..
 3. Ensure efficient data packet handling with low latency and high throughput.

Coverage:

The project will cover the design, development, and implementation of the proposed method for video manipulation and compilation. It will include the creation of algorithms, system architecture, and hardware standardization guidelines. The

scope also encompasses testing, evaluation, and optimization of the system for efficient video processing.

- Limitations:
 1. The system may face challenges related to hardware compatibility and scalability
 2. It may require significant expertise in diverse domains and resources for implementation.
 3. The scope does not include the development of specific applications or services for end-users but focuses on the underlying technology for big video processing.

2. Literature Survey or Existing System

1.Big data analytics for MOOC video watching behavior based on Spark

(Hui Hu, Guofeng Zhang, Wanlin Gao, Minjuan Wang)

Objective: The study evaluates the effectiveness of MOOCs at China Agricultural University by analyzing students' video watching behavior, employing Spark-based algorithms for data preprocessing and the entropy weight method for analysis.

Existing Approach: A big data analysis framework is proposed, utilizing detailed log models of student behavior and Apache Spark for efficient analysis. It focuses on video watching behavior from the cauX platform, employing feature selection techniques and error analysis to enhance data quality.

Results: Experimental findings highlight morning peaks in video watching, with specific actions exhibiting varying trends. The entropy weight analysis emphasizes the concentration of video watching behaviors within the first 360 seconds.

Advantages: Utilizes Spark to analyze large-scale MOOC log data, offering insights into granular video interactions and determining the relative importance of different behaviors. Aims to produce practical recommendations for course design and MOOC platform development.

Limitations: Limited by focusing on data from a single course, potentially impacting generalization. The analysis primarily focuses on video behavior, neglecting other aspects of MOOC engagement. Technical complexity may pose challenges for educators or MOOC owners without a strong data science background

2. Real-Time Big Data Delivery in Wireless Networks: A Case Study on Video Delivery

(Xu Zheng and Zhipeng Cai)

Objective: The paper addresses the challenge of real-time big data delivery in wireless networks, specifically focusing on video streaming scenarios. It introduces the Buffered Optimistic Scheduling with Rebuffering (BOSR) method to optimize resource allocation and enhance user experience.

Existing Approach: The study critiques current packet scheduling policies in wireless networks, highlighting their limitations in handling real-time big data. It discusses the insufficiency of existing techniques like Apache Storm and map-reduce extensions, emphasizing the need for a new approach.

Results: Simulation outcomes demonstrate that BOSR outperforms existing methods by serving a higher number of video requests, achieving better service ratios, and reducing failed requests under varying traffic loads. BOSR effectively handles fluctuating traffic and varying channel conditions while maintaining an acceptable user experience.

Advantages: BOSR increases capacity compared to traditional scheduling methods, reduces rebuffering events, and has potential for adaptation to scenarios with heterogeneous channel conditions.

Limitations: The computational complexity of finding the optimal scheduling solution is high, and BOSR's performance can be sensitive to parameter tuning, requiring careful adjustment for real-world scenarios.

3. Video Transformation in Big Video Era and Its Impact on Content Editing

(Mingzhi Yin)

Objective: The paper investigates the impact of the "Big Video Era" on video content editing techniques, exploring trends, challenges, and implications for creators and editors.

Existing Approach: Utilizing a literature review approach, the paper gathers insights from various scholarly articles and research reports to analyze trends in video production, consumption patterns, and technological advancements. It discusses the rise of short-form video platforms like TikTok and addresses challenges such as misinformation and content homogenization.

Results: The paper identifies both positive and negative impacts of the Big Video Era, including increased public participation in video production and challenges related to quality control and value orientation. It outlines new requirements for video content editing, emphasizing the importance of storytelling, values alignment, and creativity.

Advantages: The paper highlights key trends in the Big Video Era, provides insights for adapting editing techniques, and raises awareness about potential issues within the video landscape.

Limitations: The scope of the paper may be broad, and focusing on specific aspects of the Big Video Era could enhance effectiveness. Additionally, the rapid evolution of technology and video trends requires the analysis to stay up-to-date.

4.Scanner: Efficient Video Analysis at Scale

(Alex Poms, Will Crichton, Pat Hanrahan, Kayvon Fatahalian)

Objective: The paper introduces Scanner, a system designed for efficient, large-scale video analysis, addressing challenges in handling large video datasets and optimizing processing efficiency.

Existing Approach: The paper discusses challenges in scaling existing video processing algorithms, including long processing times, resource limitations, and lack of parallelism. It highlights the need for more efficient methods to handle large video datasets.

Results: The paper presents results comparing Scanner-based implementations with previous approaches for tasks such as 3D pose reconstruction, hyperlapse generation, and VR video stitching. It demonstrates improvements in processing times, scalability, and efficiency achieved by Scanner across different hardware configurations.

Advantages:

Optimized Data Management: Scanner utilizes a logical table representation for efficient storage and retrieval of video data.

Flexible Processing: Its dataflow graph structure allows customization of analysis pipelines, enhancing flexibility.

Hardware Efficiency: Scanner can work with CPUs and GPUs, optimizing task distribution for improved performance.

Scalability: Designed for large video datasets, Scanner can utilize multiple machines for distributed processing, ensuring scalability.

Limitations:

Complexity: Large-scale video analysis requires intricate scheduling and resource management, posing challenges.

Temporal Dependencies: Maintaining context across multiple frames in a video can be computationally demanding, impacting efficiency.

3. Product Perspective

The product originated from the need for a fast and efficient method for video manipulation and compilation using Big Data Technologies. The context of the product is the increasing demand for high-quality video processing in various industries such as entertainment, education, and surveillance. Traditional video processing methods often struggle to keep up with the demands of processing large video files and real-time streaming. Therefore, the product was conceptualized to address these challenges and provide a solution that leverages Big Data Technologies to improve video processing speed, efficiency, and quality.

3.1. Product Features

1. **Parallel Video Processing:** Utilizes multi-core CPUs for parallel processing, enabling faster video processing.
2. **GPU Acceleration:** Utilizes GPUs for video processing tasks such as rendering, encoding, and decoding, further enhancing processing speed.
3. **Efficient Memory Management:** Handles large frame sizes efficiently during processing using various memory types like RAM, DDR4, DDR5.
4. **Scalability:** Designed to be scalable, allowing for the addition of more processing nodes to handle increasing processing demands.
5. **Compatibility:** Compatible with a wide range of video formats and codecs, ensuring flexibility in video processing tasks.

3.2. User Classes and Characteristics

[Anticipate various user classes that will use this product. User classes may be distinct based on how often they use the product, functionality of products, technical capability, security levels, and many more. Summarize each user class.]

1. Video Production Professionals:

- Frequency of Use: Regularly, for professional video editing and production tasks.
- Functionality: Require advanced features for editing, rendering, and processing high-quality videos.
- Technical Capability: High technical proficiency in video editing software and hardware.
- Security Levels: Need secure storage and access to sensitive video content.

2. Educators and Trainers:

- Frequency of Use: Regularly, for creating educational videos and training materials.
- Functionality: Need tools for recording, editing, and sharing instructional videos.
- Technical Capability: Moderate technical skills, with a focus on usability and simplicity.
- Security Levels: Require secure access to educational content and student data.

3. Streaming Platform Administrators:

- Frequency of Use: Daily, for managing and optimizing video streaming services.
- Functionality: Require tools for monitoring, analyzing, and improving video streaming performance.
- Technical Capability: High technical proficiency in managing streaming servers and networks.
- Security Levels: Need secure access to streaming platform data and user information.

3.3. Operating Environment

The system will operate in an environment based on Linux distributions, specifically Ubuntu 22.05. This choice of operating system provides a stable and reliable platform for the system's operations. Ubuntu 22.05 offers a user-friendly interface and robust security features, making it well-suited for various applications, including video processing.

3.4. General Constraints, Assumptions and Dependencies

1. **Hardware Limitations:** Signal timing requirements and limitations of hardware components, such as multi-core CPUs and GPUs, may constrain the choices for system design and implementation.
2. **Limitations of Simulation Programs:** Constraints imposed by the capabilities and limitations of simulation programs used for testing and development may impact the choices available to developers.
3. **Parallel Operations:** The need for parallel processing and its constraints, such as synchronization and communication overhead, may influence the design and implementation choices.

3.5. Risks

1. **Resource Availability:** There may be a risk of resource scarcity, such as limited availability of multi-core CPUs or GPUs, which could affect the system's performance and scalability.
2. **Compatibility Issues:** There is a risk of compatibility issues with different hardware components and software libraries, which could lead to system instability or functionality limitations.
3. **Technical Complexity:** The technical complexity of implementing parallel processing and GPU acceleration could lead to delays or difficulties in development.
4. **Performance Optimization:** Achieving optimal performance with parallel processing and GPU acceleration may require extensive testing and tuning, posing risks to project timelines.
5. **Scalability Challenges:** Ensuring the system can scale effectively to handle increasing processing demands without compromising performance or stability is a potential risk.

4. Functional Requirements

Validity tests on inputs:

- Validation of input video for compatibility, and codec using a predefined format. check for metadata and necessary data required for processing (example: timestamp, camera device details maybe)

Sequence of operations:

- Workload distribution across cluster of machines based on workload distribution techniques (big data frameworks and cluster formation)
- Create and set up video pipeline and split them to phases/stages

Data ingestion,pre-processing:

- Use distributed computing frameworks like apache spark to parallelise computation tasks across nodes in a cluster. Fault tolerance and continual processing

Error handling and recovery:

- Error handling mechanisms, logging and monitoring, set up interface/provisions for manual intervention

Consequences of parameters:

- resource utilization, improvise this by adjusting params like memory allocation, and partitioning of data
- include performance metrics.

Relationship of outputs to inputs:

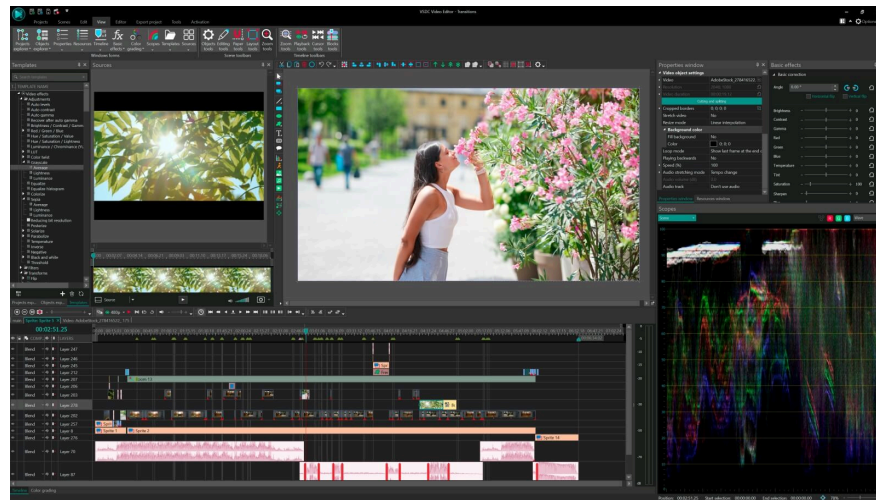
- Video generation (output containing the desired modification/transformation, enhancements edits)
- Reassociate metadata and necessary data used for processing.

5. External Interface Requirements

5.1. User Interfaces

[Describe the logical characteristics of the interface connecting the system and the users. This may include,

- Required screen formats with GUI standards for styles.
- Screen layout and standard functions (e.g. help).
- Relative timing of inputs and outputs.
- Availability of some form of programmable function key.
- Error messages.]



5.2. Hardware Requirements

1. Multi-core CPUs are required for parallel processing.
2. GPUs are necessary for video processing tasks such as rendering, encoding, and decoding.
3. Memory, including RAM, DDR4, DDR5, is needed to handle large frame sizes efficiently during processing.
4. Considerable storage with high-speed data access rates is necessary for handling large video files and temporary data storage during processing.
5. Networking infrastructure includes Ethernet interfaces, network switches, routers, and equipment for communication between distributed computing nodes (cluster creation).
6. Regular power supply, including UPS, is needed to keep the systems running in case of loads that take numerous hours to run

5.3. Software Requirements

- Apache Spark:
Apache Spark is a distributed computing framework designed for large-scale data processing tasks, including batch processing, streaming, machine learning, and graph processing.
- Apache Hadoop HDFS
- Tools: Code Editors, SSH and Remote Access, Video Processing Libraries, Monitoring and Logging Tools.
- Languages: Python
- Version Control Systems (Git)

- Operating System:
Linux distro's
Ubuntu (22.04 LTS)

5.4. Communication Interfaces

Interfaces Required for Communication:

1. Local Area Network (LAN) Protocols: The system may utilize LAN protocols such as Ethernet for communication between devices within the same network. This allows for high-speed data transfer between nodes in the system.
2. Line Speed: The system should support high line speeds to ensure efficient data transfer, especially for real-time video processing. The line speed should be sufficient to handle the bandwidth requirements of the video data.
3. Buffer Size: A large buffer size is required to store incoming video data temporarily before processing. This helps in managing the data flow and ensures smooth processing without data loss or delay.

6. Non-Functional Requirements

6.1. Performance Requirement

1. Processing Speed: The system must process video manipulation tasks efficiently, ensuring that edits, including image replacements and additional effects, are applied swiftly to maintain user productivity.
2. Scalability: The system should scale seamlessly with increasing demand, allowing it to handle a growing number of concurrent video editing tasks without degradation in performance.
3. Resource Utilization: Optimize resource utilization, including CPU, memory, and storage, to ensure efficient processing and minimize latency during video editing operations.

6.2. Safety Requirements

1. User Safety: Implement safeguards to prevent accidental deletion or corruption of original video files during editing operations. Provide confirmation prompts and undo functionalities to mitigate the risk of irreversible changes.
2. Error Handling: Design robust error handling mechanisms to detect and recover from errors safely, minimizing the impact on user data and system stability.
3. Resource Management: Manage system resources effectively to prevent resource exhaustion and maintain system stability. Implement throttling mechanisms to prevent overload and prioritize critical tasks.

6.3. Security Requirements

1. Access Control: Implement role-based access control (RBAC) mechanisms to restrict access to editing functionalities based on user roles and permissions. Enforce strong password policies to prevent unauthorized access.
2. Identity Authentication: Require user authentication before granting access to the system, using secure authentication methods such as multi-factor authentication (MFA) to verify user identities.
3. Audit Logging: Maintain detailed audit logs of user activities, including login attempts, file modifications, and system configurations, to track and monitor user actions for security and compliance purposes.

7. Other Requirements

1. Portability: The system should be deployable across different environments, including on-premises servers, cloud platforms, and containers. It should be compatible with various operating systems and infrastructure configurations to enable seamless deployment and migration.
2. Usability: The system should offer a user-friendly interface that is intuitive and easy to navigate. It should provide clear instructions and feedback to guide users through the video editing process, minimizing the learning curve and enhancing user satisfaction.
3. Performance Monitoring: Implement performance monitoring tools and metrics to track the system's performance in real-time. This includes monitoring resource utilization, processing times, and system availability to identify and address performance bottlenecks proactively.
4. Efficiency: Optimize resource usage and operational costs to ensure cost-effectiveness in deploying and maintaining the system. This includes implementing resource allocation strategies and optimization techniques to minimize infrastructure expenses while maximizing performance.

Appendix A: Definitions, Acronyms and Abbreviations

Definitions:

1. Video Manipulation: The process of altering or modifying video content, including editing, adding effects, and adjusting visuals or audio elements.
2. Video Compilation: The act of combining multiple video clips or segments into a single cohesive video production.
3. Big Data Technologies: Technologies and methodologies used for processing, storing, and analyzing large volumes of data, typically characterized by the three Vs: volume, velocity, and variety.
4. Algorithm: A set of rules or instructions designed to perform a specific task or solve a particular problem, often used in video processing and manipulation.

5. Data Processing: The conversion or manipulation of raw data into a more meaningful form, often involving tasks such as filtering, sorting, and analyzing.
6. Scalability: The ability of a system to handle increasing amounts of data or workload by adding resources or nodes without sacrificing performance.
7. Latency: The delay between initiating a task and receiving a response, often a critical factor in video editing operations.
8. Reliability: The consistency and dependability of a system in executing task accurately and without errors.
9. Robustness: The ability of a system to maintain stability and performance even in the face of unexpected inputs or adverse conditions.
10. Availability: The extent to which a system remains operational and accessible to users, typically measured by uptime and downtime.
11. Encryption: The process of encoding data to make it unreadable to unauthorized users, often used to secure sensitive information during transmission or storage.

Acronyms and Abbreviations:

1. RBAC: Role-Based Access Control
2. MFA: Multi-Factor Authentication
3. ACID: Atomicity, Consistency, Isolation, Durability
4. CDN: Content Delivery Network
5. API: Application Programming Interface
6. CPU: Central Processing Unit
7. GPU: Graphics Processing Unit
8. I/O: Input/Output
9. DNS: Domain Name System
10. HTML: Hypertext Markup Language
11. CSS: Cascading Style Sheets
12. JSON: JavaScript Object Notation
13. XML: Extensible Markup Language
14. REST: Representational State Transfer

Appendix B: References

1. <https://link.springer.com/article/10.1007/s00521-018-03983-z>
2. <https://ieeexplore.ieee.org/abstract/document/7874183>
3. <https://www.scirp.org/journal/paperinformation?paperid=113330>
4. <https://arxiv.org/abs/1805.07339>
5. <https://spark.apache.org/>
6. <https://hadoop.apache.org/>
7. https://hadoop.apache.org/docs/r1.2.1/hdfs_design.html
8. <https://www.techtarget.com/searchstorage/essentialguide/Choosing-storage-for-streaming-large-files-in-big-data-sets>